

An FPGA-Based Gomoku Game with VGA Display, PS/2 Mouse Input, and Heuristic AI

Zeng Dingvao

Han Yuzhou

Jin Wenyuan

Digital System Design Project

Digital System Design Project

Digital System Design Project

Jinge Guo

Digital System Design Project

Abstract—This report presents the design and implementation of a Gomoku game system on an FPGA using VHDL. The project targets an interactive hardware game rather than a software-rendered interface: the top-level design produces a 640×480 VGA video stream, reads a PS/2 mouse as the pointing device, maintains a 15×15 game board, supports player-versus-player and player-versus-AI modes, and detects the winning line after five consecutive stones are formed. The received Vivado project uses `vga_ctrl` as the active top module and integrates a game controller, a PS/2 mouse controller, a low-level PS/2 interface, and an AI controller. The game controller implements mode selection, move validation, turn switching, undo, reset, board-state storage, and win detection. The AI controller evaluates all empty cells using a directional scoring function that considers attack, defense, and center preference. Implementation results show successful synthesis, placement, routing, and bitstream generation for the Artix-7 device xc7a100tcsg324-1. Vivado reports indicate 25,144 placed slice LUTs, 1,008 slice registers, 36 DSP blocks, no block RAM usage, estimated on-chip power of 0.513 W, and a routed timing summary with WNS of 8.226 ns. The design therefore demonstrates a complete FPGA-oriented game architecture with real-time video output and hardware-managed game logic.

Index Terms—FPGA, VHDL, Gomoku, VGA, PS/2 mouse, game controller, heuristic AI

I. Introduction

Board games are useful digital-system design targets because they require a combination of control logic, state storage, user input, output timing, and correctness checking. Gomoku, also known as five-in-a-row, is especially suitable for an FPGA project because the game rules are simple enough to implement in synchronous logic, while the system-level design still includes nontrivial hardware problems such as video timing, input decoding, board-memory representation, and win-condition detection.

The objective of this project is to build an FPGA-based Gomoku game using VHDL. The report is formatted with the IEEE conference template [1]; the hardware description follows VHDL design practice [2]; and the implementation flow is based on Vivado synthesis, implementation, timing, and power reports [3]. Compared with a software implementation, the design must explicitly describe how each part of the system is clocked, how each state update is

triggered, and how video pixels are produced in real time. The updated project moves beyond the earlier push-button prototype by using a PS/2 mouse for interaction and by rendering the complete interface directly through VGA. The final system includes a mode-selection screen, a 15×15 board, black and white pieces, a mouse cursor, a hover indicator, a red winning line, player-versus-player (PvP) mode, player-versus-environment (PvE) mode, undo by right click, and reset by middle click.

The rest of this paper is organized as follows. Section II describes the architecture, hardware modules, data path, control path, and key VHDL implementation. Section III reports synthesis and implementation results and analyzes correctness, timing, resource usage, power, and limitations. Section IV concludes the work and gives possible extensions.

II. Methodology

A. System Architecture

The Vivado project sets `vga_ctrl` as the active top module. Its external ports are the FPGA clock input, VGA horizontal and vertical synchronization, 4-bit red/green/blue color channels, and bidirectional PS/2 clock and data lines. Internally, the design uses a divided VGA/game clock, VGA scan counters, a PS/2 mouse subsystem, a central game controller, and an AI controller.

Fig. 1 summarizes the implemented architecture. The PS/2 mouse controller converts serial mouse packets into cursor coordinates and button states. The top-level VGA controller converts mouse coordinates into board-grid coordinates and edge-detects mouse clicks. The game controller stores the board, updates turns, handles mode selection, checks victory, and requests AI moves. The AI controller scans legal moves and returns the highest-scoring candidate. The VGA pixel renderer reads the current board and game state every pixel cycle and produces the displayed image.

B. Hardware Design

The display subsystem follows the standard 640×480 VGA timing structure. The active region

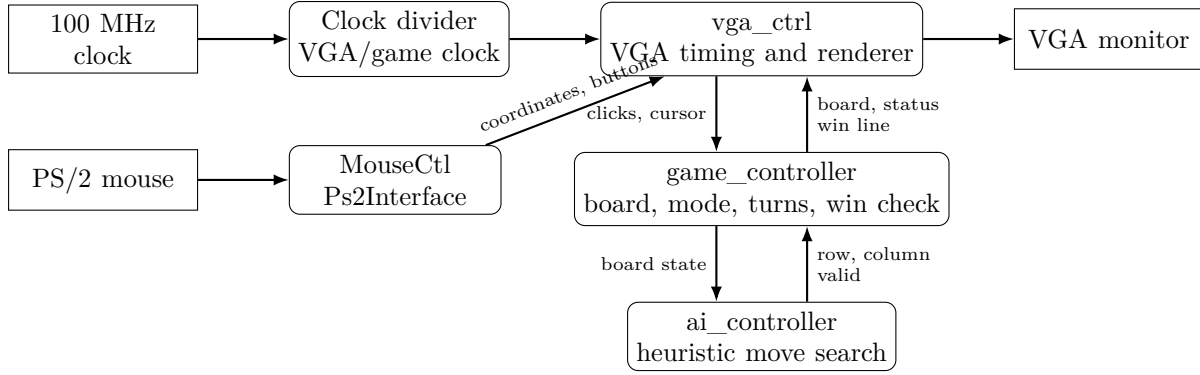


Fig. 1. Block diagram of the FPGA Gomoku system.

is defined by horizontal and vertical counters with $H_ACTIVE=640$, $H_TOTAL=800$, $V_ACTIVE=480$, and $V_TOTAL=525$. The board is drawn as a 15×15 grid. Each cell uses a 28-pixel playable region and a 2-pixel grid line, giving a compact board centered in the active VGA frame.

The game state is represented by two 225-bit vectors. `board_pieces` records whether each grid location is occupied, while `board_colors` records the corresponding stone color. This split representation keeps occupancy checks and color checks simple. The index mapping is

$$idx = y \times 15 + x, \quad (1)$$

where x and y are board coordinates. The move history is stored in two integer stacks with up to 100 recorded moves. This enables right-click undo by clearing the most recent position and toggling the player turn.

The control path is centered on a two-state game flow: `MODE_SELECT` and `PLAYING`. In mode selection, a left click inside the PvP or PvE button initializes a new board and enters gameplay. In gameplay, left click attempts to place a piece, right click triggers undo, and middle click resets the system to the mode-selection screen. In PvE mode, the human player uses color '1' and the AI uses color '0'.

C. ROM Module

In addition to the register-based board state, the updated system design includes a ROM-oriented memory configuration for storing fixed display data. The ROM is configured through the Vivado Block Memory Generator IP. As shown in Fig. 2, the memory uses a native interface and a single-port ROM organization. This setting is appropriate for read-only graphical data because the VGA renderer only needs deterministic read access during display generation.

The selected memory parameters use a 12-bit data width and a depth of 307,200 words. This depth corresponds to a 640×480 frame, while the 12-bit word width matches the 4-bit-per-channel RGB output format used by the VGA interface. Therefore, the ROM configuration can

represent one full VGA frame or a frame-sized lookup table for static visual content. The configuration also disables ECC and uses a minimum-area implementation option, which is suitable for a course project where the ROM is used as a deterministic graphics resource rather than a fault-tolerant memory subsystem.

D. VHDL Module Description

Table I summarizes the main VHDL modules used by the active build. Additional files such as `TopGobangSystem.vhd` and `display_controller.vhd` remain in the source folder, but the Vivado project configuration and synthesis log identify `vga_ctrl` as the synthesized top module.

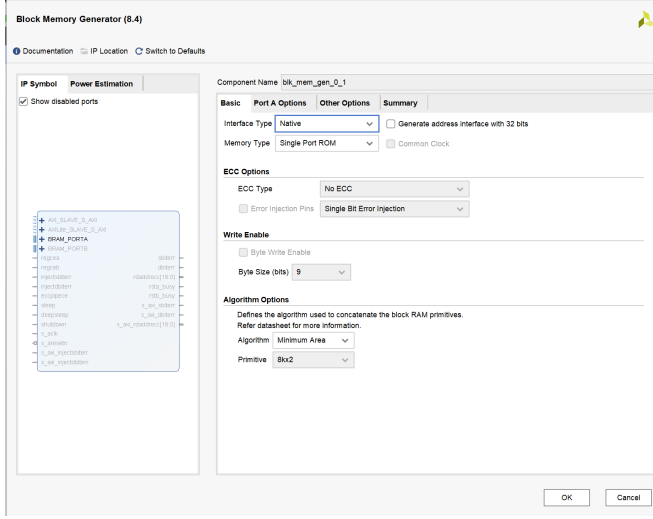
E. Key Code Explanation

The first key implementation detail is the top-level integration. The active top module instantiates the mouse controller and the game controller, while its pixel process renders either the mode-selection screen or the game board.

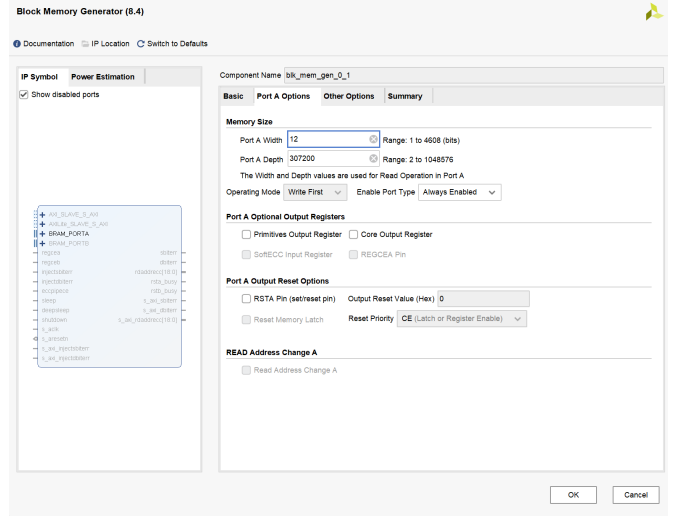
```

1 mouse_inst: MouseCtl
2 generic map(SYSCLK_FREQUENCY_HZ => 25000000)
3 port map(
4     clk => vga_clk,
5     xpos => mouse_x,
6     ypos => mouse_y,
7     left => mouse_left,
8     middle => mouse_middle,
9     right => mouse_right,
10    ps2_clk => PS2_CLK,
11    ps2_data => PS2_DATA
12);
13
14 game_ctrl_inst: game_controller
15 generic map(BSIZE => BOARD_SIZE)
16 port map(
17     clk => vga_clk,
18     mouse_x => mouse_x,
19     mouse_y => mouse_y,
20     left_click => left_click_pulse,
21     right_click => right_click_pulse,
22     middle_click => middle_click_pulse,
23     board_pieces => board_pieces,
24     board_colors => board_colors,
25     game_status => game_status,
26     win_line_valid => win_line_valid
27);

```



(a) Native single-port ROM configuration.



(b) ROM word width and depth configuration.

Fig. 2. Vivado Block Memory Generator configuration for the ROM module.

TABLE I
Major VHDL Modules in the Updated Project

Module	Function	Key Inputs	Key Outputs
vga_ctrl	Active top-level module. Generates VGA timing, renders menu/board/pieces/cursor/winning line, converts mouse position to board coordinates, and connects the mouse and game controller.	CLK_I, PS2_DATA	PS2_CLK, VGA sync and RGB signals
game_controller	Maintains board state, game mode, turn control, move history, undo/reset behavior, win detection, and AI move insertion.	Mouse coordinates, click pulses, undo/reset triggers, AI response	Board vectors, game status, winner, current player, win-line endpoints
ai_controller	Searches all empty cells and selects the best move using directional heuristic scoring.	Board occupancy, board colors, AI color, game-over flag	AI row, AI column, move-valid pulse, thinking flag
MouseCtl	Initializes and reads the PS/2 mouse; converts packets into position and button signals.	PS/2 clock/data, system clock, reset	Cursor position, left/middle/right button states
Ps2Interface	Low-level bidirectional PS/2 serial interface for transmit/receive handshaking, filtering, and frame control.	PS/2 clock/data, transmit byte, write request	Received byte, read pulse, busy/error status

Listing 1. Top-level connection between mouse input, game logic, and VGA rendering.

Listing 1 is important because it shows that the updated design is an integrated interactive system. Mouse events are not handled by external software; they are decoded and consumed by the FPGA logic.

The second key part is the game-state update. The controller validates a click by converting the mouse location into board coordinates, checking the occupancy bit, writing the piece and color vectors, and then testing whether the move ends the game.

```

1  elsif game_over = '0' and left_click = '1' then
2    gx := get_grid_x(mouse_x);
3    gy := get_grid_y(mouse_y);
4    if gx >= 0 and gy >= 0 then
5      idx := gy * BSIZE + gx;
6      if pieces(idx) = '0' then
7        pieces(idx) <= '1';
8        colors(idx) <= player_turn;
9        move_x_stack(move_cnt) <= gx;

```

```

move_y_stack(move_cnt) <= gy;
move_cnt <= move_cnt + 1;
if check_win(pieces, colors, gx, gy, player_turn) then
  game_over <= '1';
  winner <= player_turn;
  get_win_line(gx, gy, player_turn, sx, sy, ex, ey);
  win_valid_reg <= '1';
end if;
player_turn <= not player_turn;
end if;
end if;
end if;

```

Listing 2. Simplified player-move update in the game controller.

The win checker searches in four directions: horizontal, vertical, main diagonal, and anti-diagonal. For each direction it counts same-color stones on both sides of the last move. This approach is efficient because it only checks lines that can be affected by the newest move.

The AI controller is implemented as a simple finite-state machine. It scans the board over 225 cycles, evaluates empty cells, and emits one valid pulse when the best move

is ready.

```

1 when THINKING =>
2   thinking_reg <= '1';
3   if think_cnt < BSIZE*BSIZE then
4     i := think_cnt / BSIZE;
5     j := think_cnt mod BSIZE;
6     if not has_piece(board_pieces, i, j) then
7       temp_score := calc_total_score(
8         board_pieces, board_colors, i, j, ai_color);
9       if temp_score > best_score then
10        best_score := temp_score;
11        best_row_reg <= i;
12        best_col_reg <= j;
13        found := true;
14      end if;
15    end if;
16    think_cnt <= think_cnt + 1;
17  else
18    if found then
19      state <= DONE;
20    else
21      state <= IDLE;
22    end if;
23  end if;

```

Listing 3. AI scanning state machine.

The scoring function gives high priority to immediate wins and blocking opponent threats. For example, a self four-in-a-row returns 10000, while an opponent four-in-a-row returns 9000. Lower scores are assigned to open threes, semi-open threes, twos, and single-stone patterns. A center-distance term is added so that the AI favors central positions when no strong tactical move exists.

III. Results and Analysis

A. Simulation Results

The received project folder contains an active Vivado simulation set, but no dedicated testbench source, waveform configuration, or waveform screenshot was found in the submitted archive. Therefore, this report does not fabricate a simulation waveform. Functional validation is instead inferred from the VHDL structure and from Vivado synthesis/implementation results. Table II lists the functional scenarios that are directly supported by the implemented logic and should be used as simulation test cases in a future testbench.

B. Synthesis and Implementation Results

Vivado synthesis was run with `synth_design -top vga_ctrl -part xc7a100tcsq324-1`. The synthesis run completed successfully with 0 errors, 0 critical warnings, and 0 warnings. Implementation also completed successfully through placement, routing, reporting, and bitstream generation. The generated bitstream is `vga_ctrl.bit`.

The routed timing report states that all user-specified timing constraints are met. The design timing summary reports WNS of 8.226 ns, TNS of 0.000 ns, WHS of 0.302 ns, and THS of 0.000 ns. These values indicate no setup or hold failure under the current constraints. Table III summarizes the placed utilization.

TABLE II
Recommended Functional Test Cases

Test case	Expected behavior
Mode selection	A click inside the PvP or PvE button initializes the board and enters gameplay.
Legal move	A click inside an empty board cell sets the corresponding occupancy bit and records the player color.
Illegal move	A click on an occupied cell leaves the board state unchanged.
Win detection	Five consecutive same-color stones set <code>game_over</code> , <code>winner</code> , and valid win-line endpoints.
Undo	Right click clears the most recent move and toggles the current player when the move stack is nonempty.
Reset	Middle click returns to mode selection and clears game state.
AI move	In PvE mode, after the human move, the AI scans the board and places a legal response.

TABLE III
Post-Placement Resource Utilization

Resource	Used	Available	Util.
Slice LUTs	25,144	63,400	39.66%
Slice Registers	1,008	126,800	0.79%
Block RAM Tile	0	135	0.00%
DSP48E1	36	240	15.00%
BUFGCTRL	2	32	6.25%

The estimated routed on-chip power is 0.513 W, including 0.414 W dynamic power and 0.098 W static power. Logic and signal switching dominate the dynamic estimate, which is expected because the VGA renderer evaluates pixel-dependent conditions every active pixel.

C. Hardware and Practical Results

The implemented design targets the Nexys 4 DDR board family, whose Artix-7 FPGA, VGA connector, and PS/2 interface match the project ports and constraints [4], [5]. The practical expected operation is as follows. After configuration, the monitor displays a blue mode-selection interface. The mouse cursor is rendered as a white cross. Clicking the upper green button starts PvP mode, and clicking the lower green button starts PvE mode. In gameplay, left clicking a board intersection places a stone if the target cell is empty. A yellow/red hover marker identifies the cell under the cursor. When a five-in-a-row pattern is detected, the controller records the start and end coordinates of the line and the renderer overlays a red line through the winning stones.

The Vivado run proves that the design is syntactically valid, synthesizable, placeable, routable, and bitstream-generatable. The photographed VGA output in Fig. 3 further provides board-level evidence that the generated bitstream drives the intended visual interface.

D. Result Analysis

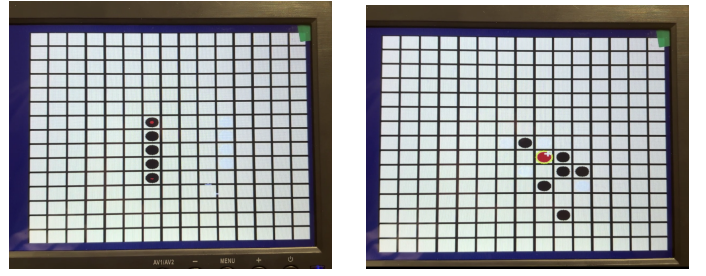
Functionally, the architecture meets the main project goal: it implements a playable Gomoku system with direct hardware rendering and mouse-based control. Compared with a minimal grid-and-button version, the updated project provides a more complete user interface and a stronger game model. The use of two bit vectors for board occupancy and color is compact and simple, and the move stack provides practical undo support.

The AI design is suitable for an FPGA course project because it avoids recursive game-tree search and instead uses a deterministic single-ply heuristic. Scanning all 225 positions over sequential cycles keeps the control structure clear. The tradeoff is that the AI is tactical rather than strategic: it can prioritize immediate threats and central positions, but it does not evaluate multi-move plans.

Resource usage is dominated by LUTs and DSP blocks. The relatively high LUT utilization is caused by pixel rendering conditions, directional win logic, and AI scoring logic. The design uses no block RAM because the board is stored in distributed registers and vectors rather than a memory primitive. This is reasonable for a 225-cell game board, although larger games or richer graphics would benefit from RAM-based frame buffers or sprite memory.

Timing is acceptable under the current Vivado constraints. Nevertheless, the implementation log includes many DSP pipelining warnings and the methodology report flags non-clocked sequential cells. These messages suggest possible improvement areas: replacing inferred wide arithmetic in pixel-distance calculations and AI scoring with pipelined arithmetic, constraining generated clocks more explicitly, and avoiding divided clocks where a clock-enable strategy would be cleaner. The bitstream still completes successfully, but these issues should be addressed before treating the design as production-quality hardware.

The practical display results confirm that the VGA output correctly presents a 15×15 grid, black stones, hover highlighting, and the visible mouse cursor. The first result shows a vertical sequence of stones and the highlighted current grid position, while the second result shows a more distributed board state with the hover marker placed near the center. These observations confirm that the video timing, board rendering, piece drawing, and mouse-position feedback operate together on the external display.



(a) Vertical move pattern. (b) Distributed board state.

Fig. 3. Practical VGA display results of the FPGA Gomoku game.

IV. Conclusion

This project implements a VHDL-based Gomoku game for an FPGA platform. The updated design uses `vga_ctrl` as the top-level module and integrates VGA timing, PS/2 mouse input, a board-state game controller, undo/reset control, PvP/PvE mode selection, win detection, and heuristic AI move generation. The design was successfully synthesized, implemented, routed, and converted into a bitstream for the Artix-7 xc7a100tcsg324-1 device. Routed reports show that timing constraints are met and that the implementation uses moderate LUT resources, very few registers, no block RAM, and 36 DSP blocks.

The main achievement is that the game is not merely simulated as software behavior; its interface and rules are expressed as hardware modules that interact through synchronous signals and are demonstrated through VGA display output. The main limitation is the absence of submitted self-checking testbench waveforms in the archive. Future work should add self-checking VHDL testbenches, capture waveform evidence for all game events, improve generated-clock constraints, pipeline wide multipliers, optimize the VGA renderer, and enhance the AI with deeper threat analysis.

References

- [1] IEEE, “Manuscript Templates for Conference Proceedings,” Online, accessed: 2026-05-30. [Online]. Available: <https://www.ieee.org/conferences/publishing/templates.html>
- [2] IEEE Standard for VHDL Language Reference Manual, IEEE, 2019. [Online]. Available: <https://standards.ieee.org/standard/1076-2019/>
- [3] Vivado Design Suite User Guide: Synthesis (UG901), AMD, 2022. [Online]. Available: <https://docs.amd.com/v/u/en-US/ug901-vivado-synthesis>
- [4] Nexys 4 DDR FPGA Board Reference Manual, Digilent, accessed: 2026-05-30. [Online]. Available: https://digilent.com/reference/_media/nexys4-ddr:nexys4ddr_rm.pdf
- [5] 7 Series FPGAs Data Sheet: Overview (DS180), Xilinx, accessed: 2026-05-30. [Online]. Available: https://docs.amd.com/v/u/en-US/ds180_7Series_Overview